

Module 7 conditioning and stability[1]

接下来 chapter 中我们将讨论 numerical analysis 中的两个 fundamental issues: Conditioning 和 Stability. Conditioning 指的是 **perturbation behavior of a mathematical problem**; 而 Stability 指的是解决这一问题的 **algorithm 的 perturbation behavior**.

Def 7.1 (problem, problem instance)

我们把一个 problem 看作是一个 function, 把 normed VS X of data map to normed VS Y of solutions. 即:

$$f : X \rightarrow Y$$

其中, 这个 problem f together with a data point x 被称为一个 **problem instance**. (比如: 输入是 x , 问题是求 x 的平方根, $f : x \mapsto \sqrt{x}$, 那么 f together with input $x = 3$ 就是一个 problem instance.)



Conditioning 研究的就就是一个 problem instance (f, x) , 其附近 solutions 的变动行为.

一个 well-conditioned problem instance 就是指, x 附近的 small perturbations 只 lead to small changes; 而 ill-conditioned problem instance 就是指, x 附近的 small perturbations 可能引起 big changes.

7.1 absolute/relative condition number of a problem

Def 7.2 (absolute condition number)

δx 表示 x 附近的一个 small perturbation, 并用

$$\delta f = f(x + \delta x) - f(x)$$

来表示 f 随之产生的变化.

定义 absolute condition number 为:

$$\hat{\kappa}(x) := \lim_{\delta \rightarrow 0} \sup_{\|\delta x\| \leq \delta} \frac{\|\delta f\|_y}{\|\delta x\|_x}$$



Remark 即, x 附近比较小的区域上, f 的值随 x 的变化的变化量.

我们发现, 这和导数的定义很相近, 就是加了一个 norm. 当然, 并不完全是. 因为这里取的是 sup. 导数未必存在, 但是这个 sup 总是存在的 (如果 count into infity 的话). absolute condition number 是一种上极限概念, 表示 x 周围的 f 变动幅度最大的值; 而它也不完全是导数值 (recall: 导数是 best linear approximation of this function near this point, 和原 function 的形状是一样的, 这里 take 的是导数的某个 matrix operator norm, determined by 我们对 y, x 使用的 norm)

具体: 我们发现, 当 δ 足够小的时候, we have:

$$\delta f \approx J(x)\delta x$$

(这里使用 approx, 但是可以严格证明.) 从而, 如果 f differentiable,

$$\hat{\kappa}(x) := \lim_{\delta \rightarrow 0} \sup_{\|\delta x\| \leq \delta} \frac{\|J(x)\delta x\|_y}{\|\delta x\|_x} = \|J(x)\|_{x \rightarrow y}$$

简写:

$$\hat{\kappa}(x) = \|J(x)\|$$

这里还有另外一个 condition number:

Def 7.3 (relative condition number)

$$\kappa(x) := \lim_{\delta \rightarrow 0} \sup_{\|\delta x\| \leq \delta} \left(\frac{\|\delta f\|}{\|f(x)\|} \bigg/ \frac{\|\delta x\|}{\|x\|} \right)$$

(recall: δf 并不是 f 的倍数而是:

$$\delta f = f(x + \delta x) - f(x)$$

即 x perturbed 后的函数值和原先的函数值的差.)



Remark 它表示的是, 对于 x 附近的 perturbation, x 相对自身的比例变化, 引发的 f 相对自身的比例变化, 的最坏情况.

对于 differentiable f , 有

$$\kappa(x) = \frac{\|J(x)\|_{x \rightarrow y}}{\|f(x)\|_y / \|x\|_x}$$

这里对于 absolute/relative condition number 有一种不严谨的记法: 我们把 $\delta x, \delta f$ 看作 infinitesimal (当然, 严格的分析里并不存在) 那么可以简写为:

$$\hat{\kappa} = \sup_{\delta x} \frac{\|\delta f\|}{\|\delta x\|}, \quad \kappa = \sup_{\delta x} \left(\frac{\|\delta f\|}{\|f(x)\|} \bigg/ \frac{\|\delta x\|}{\|x\|} \right)$$

Example 7.1 问题 1. $f: x \mapsto \frac{x}{2}$, 即把一个数取半. 那么对于任意 x 都有:

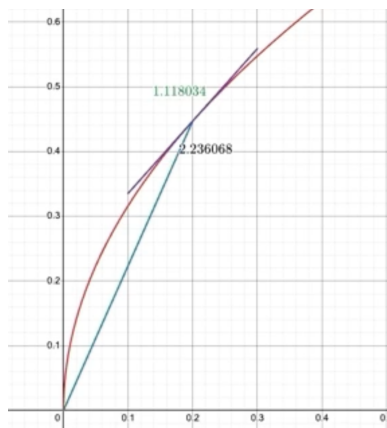
$$\kappa(x) = \frac{\|J(x)\|}{\|f(x)\|/\|x\|} = \frac{1/2}{(x/2)/x} = 1$$

well-conditioned.

问题 2. $f: x \mapsto \sqrt{x}$, 即取一个数的 sqrt, $x > 0$, 有:

$$\kappa(x) = \frac{\|J(x)\|}{\|f(x)\|/\|x\|} = \frac{1/2\sqrt{x}}{\sqrt{x}/x} = \frac{1}{2}$$

well-conditioned.



7.1.1 examples: 两数相减

Example 7.2 问题 3: $f : (x_1, x_2) \mapsto x_1 - x_2$ 两数字相减.

$$J(x) = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \right] = [1, -1]$$

For simplicity, 取 ∞ -norm, 得到 $\|J(x)\| = 2$, 于是

$$\kappa(x) = \frac{2}{|x_1 - x_2| / \max\{|x_1|, |x_2|\}}$$

如果 $|x_1 - x_2|$ large 时, κ 就会变的很大. 因而 **this problem is ill-conditioned when $x_1 \approx x_2$** . 这符合 "cancellation error": 相近的两个数相减会损失有效数字, 放大误差.

For example:

$$a = 123456.789012, \quad b = 123456.789011$$

它们的差:

$$a - b = 0.000001$$

如果浮点数只能保留 7 位有效数字 (单精度), 那么 a 被存为 123456.8, b 被存为 123456.8, 相减后结果是 0.0, 完全错误. 这就是 cancellation error: 由于精度丢失导致的小差值计算结果失真.

7.1.2 example: polynomial 求根

Example 7.3 问题 4: polynomial 求根. $f : \mathbb{C}^n \rightarrow \mathbb{C}^n$, 把 n 个系数 maps to n 个 roots.

我们考虑

$$p(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n = \sum_{k=0}^n a_k x^k$$

如果 coefficient a_i 被 perturbed by an infinitesimal quantity δa_i , 那么 the perturbation of root x_j 是多少? 答案是:

$$\delta x_j = -(\delta a_i) x_j^i / p'(x_j)$$

从而对于这个问题:

$$\kappa_j(a_i) = \frac{|\delta x_j|}{|x_j|} \bigg/ \frac{|\delta a_i|}{|a_i|} = \frac{|a_i x_j^{i-1}|}{|p'(x_j)|}$$

证明 7.3: (非 rigorous)

perturbed polynomial 即:

$$\tilde{p}(x) = p(x) + \delta a_i x^i$$

我们要求的 perturbation δx_j , 无法直接得到等式关系. 但是我们知道新的 root 是: $x_j + \delta x_j$.

即:

$$\tilde{p}(x_j + \delta x_j) = 0$$

从而:

$$p(x_j + \delta x_j) + \delta a_i (x_j + \delta x_j)^i = 0$$

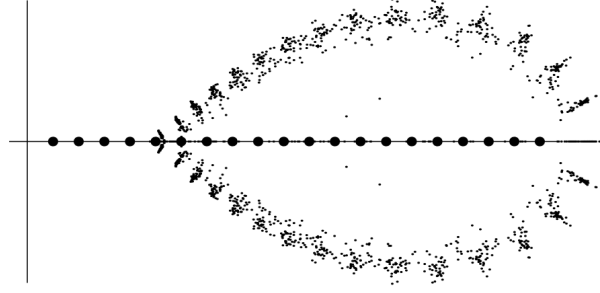


Figure 7.1: Wilkinson's example 中 roots 的 perturbation, by $\tilde{a}_k = a_k (1 + 10^{-10} r_k)$

Using Taylor expansions:

$$p(x_j + \delta x_j) \approx p(x_j) + p'(x_j) \delta x_j$$

其中 $p(x_j) = 0$. 并且, $(x_j + \delta x_j)^i \approx x_j^i$, 因为在乘方的作用下这个 perturbation 作用可以忽略 (作为高阶无穷小). 从而得到

$$p'(x_j) \delta x_j + \delta a_i x_j^i = 0$$

从而得到

$$\delta x_j = -\frac{\delta a_i x_j^i}{p'(x_j)}$$

Polynomial rootfinding 是 ill-conditioned, 即便不涉及 multiple roots 问题. 比如经典的 "Wilkinson polynomial":

$$p(x) = \prod_{i=1}^{20} (x - i) = a_0 + a_1 x + \cdots + a_{19} x^{19} + x^{20}$$

它的 most sensitive root 是 $x = 15$, 并且对于这个 root, 最 sensitive 的 coefficient to change 是 $a_{15} \approx 1.67 \times 10^9$, 这个 root 和这个 coeff 之间的 condition number 为:

$$\kappa \approx \frac{1.67 \times 10^9 \cdot 15^{14}}{5!14!} \approx 5.1 \times 10^{13}$$

7.1.3 example: matrix 乘 vector

这个 example 分为三部分:

For problem $Ax = b$:

1. Fixing A , $x \mapsto b$
2. Fixing A , inverse problem: $b \mapsto x$
3. fixing b , $A \mapsto x$

Example 7.4 Matrix-vector multiplication: $Ax = b$ (fixing A)

$$\kappa_{\text{fwd}}(x) = \sup_{\delta x} \left(\frac{\|A(x + \delta x) - Ax\|}{\|Ax\|} / \frac{\|\delta x\|}{\|x\|} \right) = \sup_{\delta x} \frac{\|A\delta x\|}{\|\delta x\|} / \frac{\|Ax\|}{\|x\|}$$

即:

$$\kappa_{\text{fwd}}(x) = \|A\| \frac{\|x\|}{\|Ax\|}$$

Note: 对于任意非零 x , 都有

$$\|Ax\| \geq \frac{1}{\|A^\dagger\|} \cdot \|x\| \implies \frac{\|x\|}{\|Ax\|} \leq \|A^\dagger\|$$

所以

$$\kappa_{\text{fwd}}(x) = \|A\| \cdot \frac{\|x\|}{\|Ax\|} \leq \|A\| \|A^\dagger\|$$

(2) Inverse problem: given b 求 x , 即 $x = A^{-1}b$, 也有同样的 condition number bound (这显然, 因为对称):

$$\kappa_{\text{inverse}}(b) = \|A^\dagger\| \cdot \frac{\|b\|}{\|x\|} \leq \|A\| \|A^\dagger\|$$

因而我们把 $\|A\| \|A^{-1}\|$ 称为一个 matrix 的 condition number.

Def 7.4 (condition number of a matrix)

我们定义 condition number of a matrix:

$$\kappa(A) := \|A\| \|A^\dagger\|$$

Notice: 如果 $\|\cdot\| := \|\cdot\|_2$, 那么 for $A \in \mathbb{C}^{m \times n}$, we have:

$$\|A\|_2 = \sigma_1, \quad \|A^\dagger\|_2 = 1/\sigma_r$$

where σ_r 是最小的非零的 singular value.



对于 $\|\cdot\| := \|\cdot\|_2$,

$$\kappa(A) = \frac{\sigma_1}{\sigma_r}$$

即 image of the unit sphere 作为 hyperellipse 的 eccentricity.

Remark 我们已经知道了, 对于 $Ax = b$ ($A \in \mathbb{C}^{m \times n}$): 不论是 forward problem $x \mapsto b$ 还是 inverse problem $b \mapsto x$, 都有

$$\kappa \leq \kappa(A)$$

那么等号在什么时候取到呢? 我们以 forward problem 为例: 取 A 的 SVD: $A = U\Sigma V^*$, where

- $U \in \mathbb{C}^{m \times m}, V \in \mathbb{C}^{n \times n}$ unitary,
- $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_r, 0, \dots, 0) \in \mathbb{R}^{m \times n}, r = \text{rank}(A)$
- $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$

我们令 $x = Vz$, 因为 V unitary 不改变范数,

$$Ax = U\Sigma V^* Vz = U\Sigma z \Rightarrow \|Ax\| = \|\Sigma z\|, \quad \|x\| = \|z\|$$

所以

$$\frac{\|x\|}{\|Ax\|} = \frac{\|z\|}{\|\Sigma z\|} = \frac{\left(\sum_{i=1}^n |z_i|^2\right)^{1/2}}{\left(\sum_{i=1}^r \sigma_i^2 |z_i|^2\right)^{1/2}}$$

这个最大值就是:

$$\max_{z \neq 0} \frac{\|z\|}{\|\Sigma z\|} = \max_{z \neq 0} \frac{1}{\left(\sum_{i=1}^r \sigma_i^2 \cdot \frac{|z_i|^2}{\|z\|^2}\right)^{1/2}}$$

这在 $|z_i| = 1$ 只有一个非零坐标, 且对应于最小非零奇异值 σ_r 时最大. 所以最大值是 $1/\sigma_r = \|A^\dagger\|$. 因此:

$$\frac{\|x\|}{\|Ax\|} = \|A^\dagger\| \iff x \in \text{span}(v_r)$$

即, 当 x 是 A 的 **minimal nonzero singular value** σ_r 的 **right singular vector** 时, 取到

$$\kappa_{\text{wd}}(x) = \kappa(A)$$

同理, 对于 inverse problem, 令 $b = Uy$, 有:

$$A^\dagger b = V\Sigma^\dagger U^* Uy = V\Sigma^\dagger y \Rightarrow \|A^\dagger b\| = \|\Sigma^\dagger y\|, \quad \|b\| = \|y\|$$

其中 $\Sigma^\dagger = \text{diag}(1/\sigma_1, \dots, 1/\sigma_r, 0, \dots, 0)$. 所以:

$$\frac{\|b\|}{\|A^\dagger b\|} = \frac{\|y\|}{\|\Sigma^\dagger y\|} = \left(\frac{\sum_{i=1}^m |y_i|^2}{\sum_{i=1}^r \frac{1}{\sigma_i^2} |y_i|^2} \right)^{1/2}$$

最大值发生在 $y = e_1$ 对应于最大奇异值 σ_1 , 此时

$$\|A^\dagger b\| = \frac{1}{\sigma_1} \|b\| \Rightarrow \frac{\|b\|}{\|A^\dagger b\|} = \sigma_1 = \|A\|$$

因此

$$\kappa_{\text{inv}}(b) = \|A^\dagger\| \cdot \|A\| \iff b \in \text{span}(u_1)$$

即, 当 b 是 A 的 **maximal singular value** σ_1 的 **left singular vector** 时, 取到

$$\kappa_{\text{inverse}}(b) = \kappa(A)$$

至此, 我们可以总结这个 theorem (a general version of Theorem 12.2 in textbook Ch12):

Theorem 7.1 (conditioning of matrix times vector)

For problem $Ax = b$ fixing A , 不论是 $x \mapsto b$ 的 forward problem 还是 $b \mapsto x$ 的 inverse problem, 都有:

$$\kappa \leq \kappa(A) := \|A\| \|A^\dagger\|$$

并且, 对于 forward problem, 等号当且仅当 x 是 A 的 minimal nonzero singular value σ_r 的 right singular vector 时取到; 对于 inverse problem, 等号当且仅当 b 是 A 的 maximal singular value σ_1 的 left singular vector 时取到.



现在我们来考虑: Fixing b , 求 $A \mapsto x$ 的问题.

我们有:

$$(A + \delta A)(x + \delta x) = b$$

我们知道 $Ax = b$, 并且可以 drop the doubly infinitesimal term $(\delta A)(\delta x)$, 从而得到

$$(\delta A)x + A(\delta x) = 0$$

即

$$\delta x = -A^\dagger(\delta A)x$$

By matrix norm 小于等于拆分后 norms 的乘积的定理, 我们于是有:

$$\|\delta x\| \leq \|A^\dagger\| \|\delta A\| \|x\|$$

即

$$\frac{\|\delta x\|}{\|x\|} / \frac{\|\delta A\|}{\|A\|} \leq \|A^\dagger\| \|A\| = \kappa(A)$$

于是我们得到

$$\kappa_{A \mapsto x} \leq \kappa(A)$$

神奇地发现, 它也被 $\kappa(A)$ bound.

并且, equality in this bound will hold whenever δA is such that

$$\|A^\dagger(\delta A)x\| = \|A^\dagger\| \|\delta A\| \|x\|$$

而, 我们可以发现对于任意 A, b , 这个 δA 一定存在, 即等号一定可以取到. 这是因为 operator norm 与其 dual norm 的等价性:

$$L : \delta A \mapsto A^\dagger(\delta A)x$$

是一个从 $\mathbb{C}^{m \times n} \rightarrow \mathbb{C}^n$ 的线性算子, 它的 operator norm 是:

$$\|L\| = \sup_{\delta A \neq 0} \frac{\|A^\dagger(\delta A)x\|}{\|\delta A\|}$$

我们可以证明这个 supremum 可以达到. 选择

$$\delta A = uv^*$$

其中 $u \in \mathbb{C}^m$ 是使得 $\|A^\dagger u\| = \|A^\dagger\|$ 的单位向量, $v = \frac{x}{\|x\|}$ 是单位方向向量. 于是:

$$(\delta A)x = (uv^*)x = u \cdot (v^*x) = u \cdot \|x\| \Rightarrow A^\dagger(\delta A)x = \|x\| \cdot A^\dagger u \Rightarrow \|A^\dagger(\delta A)x\| = \|x\| \cdot \|A^\dagger u\| = \|x\| \cdot \|A^\dagger\|$$

从而我们可以得到这个结论:

Theorem 7.2 (conditioning of matrix times vector: given b , problem $A \mapsto x$)

对于 $Ax = b$ fixing b , 考虑 problem $A \mapsto x$, 这一问题一定有 condition number:

$$\kappa = \kappa(A)$$



7.2 float number and machine epsilon

7.2.1 float number system

我们知道计算机处理的是离散的数值. 即, 一个 computer 的 number system 并非 $\mathbb{R}$ 而是 $\mathbb{R}$ 的一个 discrete (and finite, 但是 ideally 可以看作 infinite) subset $\mathbf{F}$, 称之为 float number system.

这个 $\mathbf{F}$ 由这两个参数决定:

- base integer β
- precision integer t

(通常 $\beta = 2$ 即 二进制, 而 $t = 24, 53$ for IEEE single/double precision.)

precision 决定了这个系统的对数字表示的相对精度 (即即将定义的 machine epsilon); biased exponent 决定了这个系统能够表示的数的范围的上下限. 从而,

$$F = \{0\} \cup \{\pm(\frac{m}{\beta^t})\beta^e : m \in [1, \beta^t - 1] \text{ int}, e \text{ int}\}$$

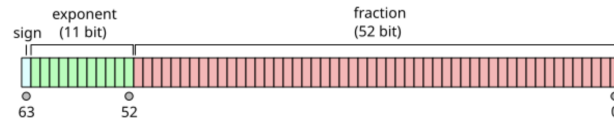


Figure 7.2: IEEE

这里的 $\pm \frac{m}{\beta^t}$ 称为 **mantissa** of x ; e 称为 **exponent**.

现实中, e 也有范围, 取决于计算机位数和架构. 比如说 ieee 双精度 float: 这里 E 的范围是 $0 \sim 2047$, 因而 $e = E - 1023$ 的范围是 $-1023 \sim 1024$. IEEE double precision:

$$x = (-1)^{\text{sign}} (1.b_{51}b_{50} \dots b_0)_2 \times 2^{E-1023}$$

因而更加现实的 system **F** 和我们这里的理论 model **F** 有这些差别:

1. 还要包括一个额外的参数: exponent offset s , 控制 e bounded by some $e_{\min}$ 和 $e_{\max}$.
2. 现实的 ieee standard 和我们的 ideal 模型 **F** 不同的点, 不仅是 e bounded 具有 $e_{\min}$ 和 $e_{\max}$, 还有: 它的每个数其实是

$$\pm(1 + \frac{m}{\beta^t})\beta^e$$

而不是 $\pm(\frac{m}{\beta^t})\beta^e$. 前面的 1 称为 **leading bit**. 这是因为在 规格化二进制浮点数系统中, 所有非零数的尾数都可以唯一表示成以 1. 开头的形式. 因为这个 1. 总是存在, 可以省略它来节省空间.

3. 考虑更多的 symbols, 例如:

Symbol	Meaning
+0	Postitive underflow; between 0 and the smallest positive representable float
-0	Negative underflow
$+\infty$	Positive overflow; bigger than biggest representable float. E.g., $1/0 = 1/(+0)$
$-\infty$	Negative overflow
NaN	Not-a-Number, e.g., $0/0$.

Note: 0 也是一个 symbol. 并且, 现实的 system 里, 还要区分正负方向上的 underflow 得到的 0.

Def 7.5 (machine epsilon)

对于一个 discrete number system **F** with precision t 和 base β , 我们定义:

$$\epsilon_{\text{machine}} := \frac{1}{2}\beta^{1-t}$$

为什么要这样定义: 因为这两点:

Proposition 7.1

对于任意的 $x \in \mathbb{R}$ that is within machine 的表示范围, 都存在一个 ϵ s.t. $|\epsilon| < \epsilon_{\text{machine}}$ 使得

$$fl(x) = x(1 + \epsilon)$$

这一点是显然的. 任意的大小不能过大的实数, 都可以在 machine epsilon 的误差内被 float number 表示. 更加好的是:

Theorem 7.3 (Fundamental Axiom of Floating Point Arithmetic)

对于一个 discrete number system $\mathbf{F}$, 对于任意的 $x, y \in \mathbf{F}$, 都存在一个 error ϵ s.t.

$$|\epsilon| \leq \epsilon_{\text{machine}}$$

such that:

$$x *_{\text{fl}} y = (x *_{\mathbb{R}} y)(1 + \epsilon)$$

for 任意的 $* := +, -, \times, \div$.

(Exclusion: relative error 并不包括 $x - x$ 时出现的 cancellation error, 以及其他的 overflow, underflow!

这些是 **symbolic hacks**, 例如 perturbing 0 to 0.1 gives relative error $\frac{0.1-0}{0} = +\infty$; 并且需要注意的是, relative errors are only useful when small, well below 100%.)



即: 任意基本运算的相对于自身的误差, 都被 bound 在 $\epsilon_{\text{machine}}$ 之内.

为什么是相对误差而不是绝对误差? 因为我们能表示的有效数字位数是固定的. 越大的数, 其小数点后的有效数字就越小. 从而, 绝对误差就越大. 但是相对误差仅和 t 和 base β 有关.

(Note: On a computer in which intermediate quantities are **truncated rather than rounded**, Fundamental Axiom of Floating Point Arithmetic hold with $\epsilon_{\text{machine}}$ **replaced by** $2\epsilon_{\text{machine}}$.)

Remark 之所以 Float number 的讨论是重要的, 因为它是计算机用来近似表示一个实数的方法, 而所有的数值计算都要经由此为媒介. 需要注意的是: **ultimate access to numbers is via $+, -, *, \div$, 复杂度和误差最终由基本运算衡量.**

对于 losing 1 bit 的 round off error 等问题, 我们并不在意; 但是, 当大量计算 iteratively 堆叠时, **一些 first algorithms for many problems might lose half the bits.** 这是一个很大的数量: 例如 Classical Gram Schmidt 会失去 half the digits, 相比 modified Gram-Schmidt 而言. 例如, accurate bits 从 52 变为 26.

因而, 基于 float number (via machine epsilon) 对一个 algorithm 的 stability 进行分析是重要的. 接下来我们将讲解 stability of an algorithm 这个概念.

7.3 stability

Review: 一个 Problem (in our def) 是一个 function $f : X \rightarrow Y$, X, Y 都是 NVS.

而我们现在定义什么是一个 algorithm:

7.3.1 def: algorithm, stability, accuracy

Def 7.6 (algorithm)

一个 algorithm for a problem $f : X \rightarrow Y$ 是另一个函数 $\tilde{f} : X \rightarrow Y$



定义上就是这么简单.

注意: 我们这里的 algorithm 是一个比较 restricted 的定义. Specially, 它并不考虑 randomized algorithms.

Example 7.5 Randomized rounding:

把 7.3 round to: 8, with a prob of 0.3; 7, with a prob of 0.7.

我们把 round 得到的结果标记为 X , 那么它则是一个 random variable. 并且, 它是一个 unbiased random variable, 即:

$$\mathbb{E}[X] = 7.3$$

这个 rounding 是一个 algorithm, 但是不包含在我们这里的定义里. 因为原问题是 $\mathbb{R} \rightarrow \mathbb{N}$ 的, 而这个问题则是 maps to random variables (我们知道一个 random variable 是一个函数, 这是一个 function space) 的. 因而它并不是我们定义的算法.

因而我们的定义其实是 restricted 的. 我们这里只考虑 deterministic 的 algorithm.

Def 7.7 (accuracy of an algorithm)

给定一个 problem f 和一个对应的 algorithm $\tilde{f}$, 我们定义 $\tilde{f}$ 的 relative error 为

$$\frac{\|\tilde{f}(x) - f(x)\|}{\|f(x)\|}$$

即: algorithm 给出的答案和正确答案的相对 difference.

如果对于每个 $x \in X$ 都有:

$$\frac{\|\tilde{f}(x) - f(x)\|}{\|f(x)\|} = O(\epsilon_{\text{machine}})$$

即 relative error is on the order of machine epsilon, 那么我们称这个 algorithm 是 accurate 的.



Problem: 对于一个 well-conditioned 的问题, 我们自然地想要一个足够 accurate 的 algorithm; 但是对于 ill-conditioned 的问题, 要求给出一个足够 accurate 的 algorithm 是很困难的事情, 因为 perturbations on ill-conditioned inputs 使得它给出准确结果的难度很大.

因而, generally, 我们应该放低要求.

Def 7.8 (stability of an algorithm)

给定一个 problem f 和一个对应的 algorithm $\tilde{f}$, 如果对于每个 $x \in X$ 都存在一个 $\tilde{x} \in X$, 其满足

$$\frac{\|\tilde{x} - x\|}{\|x\|} = O(\epsilon_{\text{machine}})$$

能够使得

$$\frac{\|\tilde{f}(x) - f(\tilde{x})\|}{\|f(\tilde{x})\|} = O(\epsilon_{\text{machine}})$$

那么则称, 这个 algorithm $\tilde{f}$ 是 stable 的.



Remark 显然, stable 是比 accurate 稍微宽松的要求. 它的要求是: **这个 algorithm gives nearly the right answer to nearly the right question.** 比起 accurate, 它放宽在于: 不需要 solve exactly the same question, 只需要 solve 一个很相近的 question 就可以了.

为什么要这样要求? 因为一个计算机很大的问题是: 我们的 input 和理论上的 input 还是不一样的. 比如, 我们要输入一个 $x = \frac{2}{3}$, 这是一个无限循环的小数, 而计算机只能输入有限位数的小数来近似. 因而关于它的计算问题, 从输入起就有了误差. 这个误差可能很小, 但是一旦遇到 ill-conditioned 的问题, 在

condition 比较差的地方 (比如说某个问题在 0 处, $\kappa(x) \rightarrow \infty$), 那么这个 input 的小误差很可能导致很大的 Output 不同.

从而, 我们需要对 input 放宽, 尽可能去容忍类似于 round up 这样的问题.

这个时候我有一个问题: stable 这个概念, 相比于 accurate, 是为了迁就 ill-conditioned 的问题, 只要保证我们的算法给出的结果一定是原问题周围某个相似的问题的数值解就可以了. 但是如果这个 problem 是 ill-conditioned, 那么原问题的解可能和它周围的其他问题的解差别非常大. 一个 stable 但不 accurate 的 algorithm, 意思就是: 在某些 ill-conditioned 的 input 上, 给出的解和我们想要的原问题的解差别非常大, 那这个算法还能成为好吗?

这就是数值分析的核心哲学问题之一. stability 已经是一个足够的条件, 因为你并不能要求算法给你 “一个问题本身都无法承诺” 的东西. Stable algorithm 是对现实的诚实反应, 承认输入的不可避免误差, 并保证: 你得到的结果是 “某个微小扰动问题” 的真实答案. 它告诉你: 在你所能拥有的误差范围内, 这就是最合理的答案了. 稳定性 = 不人为放大错误.

之所以我们只要求算法的稳定性, 就是因为: 如果非要要求准确性, 那么很多优秀的 algorithm 可能会因为仅仅几个 problem 本身就 ill-conditioned 的点上的大误差, 被判为 inaccurate.

还有一个比 stable 更强的定义

Def 7.9 (backward stability)

给定一个 problem f 和一个对应的 algorithm $\tilde{f}$, 如果对于每个 $x \in X$ 都存在一个 $\tilde{x} \in X$, 其满足

$$\frac{\|\tilde{x} - x\|}{\|x\|} = O(\epsilon_{\text{machine}})$$

能够使得 $\tilde{f}(x) = f(\tilde{x})$ 那么则称, 这个 algorithm $\tilde{f}$ 是 backward stable 的.



backward stable 的要求是: 这个 algorithm gives exactly the right answer to nearly the right question. 它蕴含的信息是: 对于这个算法 $\tilde{f}$, 任意的 output perturbation 其实都等同于某些 input perturbation. (从而可以被 input perturbation 给完全控制.)

backward stable 和 accurate 是 dual 的: accurate 要求的是这个 algorithm gives nearly the right answer to the right question.

Remark 这些定义里面的 $O(\epsilon_{\text{machine}})$ 是 across all x 的, 即存在一个 uniform bound $C\epsilon_{\text{machine}}$ among all $x \in X$, 使得这些误差值 bounded by it.

Example 7.6 我们用一个例子来阐明 “ $O(\epsilon_{\text{machine}})$ ”:

problem: 给定 b , solve system $Ax = b$ for $A \mapsto x$. 假设我们有一个 algorithm $\tilde{f} : A \mapsto x$ 是 stable 的, 那么它满足: 对于给定的 n, m , 存在 uniform bound C_1, C_2 使得对于任意的 $A \in \mathbb{C}^{n \times m}$, 都具有 nearly the same question $\tilde{A}$, 使得 algorithm $\tilde{f}$ 给出的 answer $\tilde{f}(A)$ 几乎就是这个近似问题 $\tilde{A}$ 的正确解 $f(\tilde{A})$.

Formally: 对于任意的 $A \in \mathbb{C}^{n \times m}$, 都存在 $\tilde{A} \in \mathbb{C}^{n \times m}$ s.t.

$$\frac{\|\tilde{A} - A\|}{\|A\|} \leq C_1 \epsilon_{\text{machine}}$$

使得

$$\frac{\|\tilde{f}(A) - f(\tilde{A})\|}{\|f(\tilde{A})\|} \leq C_2 \epsilon_{\text{machine}}$$

这个 C_1, C_2 是和 input 进入的 A 无关的, 它被 problem 的参数固定 (here: n, m). For example, $C_1 = 10, C_2 = 100$.

Remark 对于 finite dimensional NVS 而言, 我们不需要关注使用的是哪个 norm, 因为我们知道, finite dimensional NVS 上所有 norms 都是 topologically equiv 的. 这个等价在这里对 asymptotic bound 的讨论中很有用, 因为 topologically equiv 即: 对于任意两个 norms $\|\cdot\|_a, \|\cdot\|_b$, 都存在 C_1, C_2 使得对于任何元素 x 都有

$$C_1 \|x\|_a \leq \|x\|_b \leq C_2 \|x\|_a$$

因而, 对于 finite dimensional NVS 而言, 如果任意一个 norm 满足

$$\|x\| = O(\epsilon_{\text{machine}})$$

那么任意的 norm 都满足这一点.

7.3.2 example: floating point arithmetic

Example 7.7 当然, 四种 floating point arithmetic 是有 backward stable 的算法的.

我们以两数相减 from $f: \mathbb{C}^2 \rightarrow \mathbb{C}$ 为例: 我们 canonical 的算法就是把这两个数 round 为 float, 然后进行 float 的减法.

$$\tilde{f}(x_1, x_2) = \text{fl}(x_1) \ominus \text{fl}(x_2).$$

其中,

$$\text{fl}(x_1) = x_1(1 + \epsilon_1), \quad \text{fl}(x_2) = x_2(1 + \epsilon_2)$$

where by def, $|\epsilon_1|, |\epsilon_2| < \epsilon_{\text{machine}}$.

并且我们知道, float 减法的 error 也是 within machine epsilon 的:

$$\text{fl}(x_1) \ominus \text{fl}(x_2) = (\text{fl}(x_1) - \text{fl}(x_2))(1 + \epsilon_3)$$

从而

$$\begin{aligned} \text{fl}(x_1) \ominus \text{fl}(x_2) &= [x_1(1 + \epsilon_1) - x_2(1 + \epsilon_2)](1 + \epsilon_3) \\ &= x_1(1 + \epsilon_1)(1 + \epsilon_3) - x_2(1 + \epsilon_2)(1 + \epsilon_3) \\ &= x_1(1 + \epsilon_4) - x_2(1 + \epsilon_5) \end{aligned}$$

where

$$|\epsilon_4|, |\epsilon_5| \leq 2\epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2) = O(\epsilon_{\text{machine}})$$

我们把

$$\hat{x}_1 := x_1(1 + \epsilon_4), \quad \hat{x}_2 := x_2(1 + \epsilon_5)$$

从而, 这个 canonical algorithm 计算出的是:

$$\tilde{f}(x_1, x_2) = f(\hat{x}_1, \hat{x}_2)$$

where for all (x_1, x_2) , 它对应的这个 $(\hat{x}_1, \hat{x}_2)$ 和它在 $\mathbb{C}^2$ 中的 relative distance, within any norm 都是 $O(\epsilon_{\text{machine}})$

的.

Remark 值得提的是: 两个数相加的问题 $\mathbb{C}^2 \rightarrow \mathbb{C}$ 的 canonical algorithm 是 backward stable 的, 而一个数加一个固定的常数: $\mathbb{C} \rightarrow \mathbb{C}$ 的 canonical algorithm 却不是 backward stable 的.

7.3.3 example: inner/outer product

For inner product: problem is $f : \mathbb{C}^m \times \mathbb{C}^m \rightarrow \mathbb{R}$, given vectors $x, y \in \mathbb{C}^m$, wish to compute the inner product $\alpha = x^*y$.

显然, canonical algorithm: compute the pairwise products $\bar{x}_i y_i$ with $\otimes$ and add them with $\oplus$ to obtain a computed result $\tilde{\alpha}$. 这个算法是 **backward stable** 的.

但是, for outer product: problem is $f : \mathbb{C}^m \times \mathbb{C}^n \rightarrow \mathbb{C}^{m \times n}$.

我们想要计算 $A = xy^*$, for vectors $x \in \mathbb{C}^m, y \in \mathbb{C}^n$.

Canonical algorithm: compute the mn products $x_i \bar{y}_j$ with $\otimes$ and collect them into a matrix $\tilde{A}$.

它是 stable 的, 但却不是 backward stable 的. 因为直观而言: 我们每个 entry 的计算有不同的乘法误差, 导致: $\tilde{A}$ will 不太可能 have rank exactly 1, 因而无法真的被写作 written in the form $(x + \delta x)(y + \delta y)^*$.

Remark 对于 solution space Y 的 dimension 比 problem space X 更加大的问题 (以及 problem space 是多个输入, 其中每个输入的 space 的 dimension 比 solution space 的要小), 很少会有 backward stability.

7.3.4 theorem: what backward stability implies about the accuracy

Theorem 7.4

Suppose a backward stable algorithm is applied to solve a problem $f : X \rightarrow Y$ with condition number κ , 那么 relative errors:

$$\frac{\|\tilde{f}(x) - f(x)\|}{\|f(x)\|} = O(\kappa(x)\epsilon_{\text{machine}})$$

(notice: 这说明如果 κ of this problem bounded, 那么 backward stable algorithm 一定是 accurate 的) ♡

Proof By backward stability, we have $\tilde{f}(x) = f(\tilde{x})$ for some $\tilde{x} \in X$ satisfying

$$\frac{\|\tilde{x} - x\|}{\|x\|} = O(\epsilon_{\text{machine}})$$

我们把 $\tilde{x} - x$ 作为 δx , 从而有:

$$\frac{\|\delta x\|}{\|x\|} = O(\epsilon_{\text{machine}})$$

而由于这里 $\frac{\|\delta x\|}{\|x\|} = O(\epsilon_{\text{machine}})$ 已经是 numerically 最小的 error. 从而 By definition of $\kappa(x)$:

$$\frac{\|f(x + \delta x) - f(x)\|}{\|f(x)\|} \leq (\kappa(x) + o(1)) \cdot \frac{\|\delta x\|}{\|x\|}$$

这个不等式是因为: f 的相对变化和 x 的相对变化的比例, 其上极限就是 $\kappa(x)$. 从而 this implies

$$\frac{\|\tilde{f}(x) - f(x)\|}{\|f(x)\|} = \frac{\|f(x) - f(\tilde{x})\|}{\|f(x)\|} = \frac{\|f(x + \delta x) - f(x)\|}{\|f(x)\|} \leq (\kappa(x) + o(1)) \cdot \frac{\|\delta x\|}{\|x\|}$$

□

这一 theorem 表明: backward stability + good conditioning $\implies$ accuracy